

网络流与图论完整解答

网络流与图论问题完整解答指南

包含二分图、路径覆盖、费用流、最大权闭合图、分层图和特殊题型的详细解答，每个问题包含文字说明、伪代码和完整代码三个层次。

第一部分：基础二分图问题

1.1 二分图基础理论

定义与性质

二分图定义： 图的顶点集可以分为两个不相交的子集 (X) 和 (Y)，使得每条边的一个端点在 (X) 中，另一个端点在 (Y) 中。

关键概念：

- 匹配(Matching)** - 一个边的子集M，其中任意两条边没有公共端点
- 最大匹配** - 边数最多的匹配
- 完美匹配** - 每个顶点都被匹配的匹配
- 匹配数** - 最大匹配中的边数，记为 $|M|$

二分图判定： 图是二分图当且仅当它不含奇圈

1.1.1 二分图最大匹配的性质

König定理： 二分图中最大匹配的大小等于最小顶点覆盖的大小

$$|M| = |C|$$

推论： 二分图中最大独立集的大小为

$$|I| = |V| - |M|$$

霍尔定理： 二分图的X侧能完美匹配当且仅当对于X的任意子集S，都有

$$|N(S)| \geq |S|$$

其中 $N(S)$ 表示S的邻域。

1.2 LeetCode 1 (模拟题): 飞行员配对方案

问题描述

给定 (n) 个飞行员和 (n) 架飞机，飞行员 (i) 最擅长驾驶飞机 (j) 当且仅当 $(a[i][j] = 1)$ 。求最多能有多少个飞行员-飞机对。

特殊版本：

- 若每个飞行员和飞机都必须被配对（完美匹配），检查是否可行

文字说明

算法思想： 这是一个经典的二分图最大匹配问题。构建二分图，左侧是飞行员，右侧是飞机，如果飞行员 i 能驾驶飞机 j 则连边。使用匈牙利算法（Hungarian Algorithm）或网络流算法求最大匹配。

关键步骤：

1. 构建邻接矩阵或邻接表表示二分图
2. 对每个飞行员，使用DFS寻找增广路径
3. 增广路径是一条从未匹配飞行员开始，以未匹配飞机结束的交替路径
4. 找到增广路径后，反转路径上的匹配状态

时间复杂度： $(O(n \times m))$ ，其中 n 是飞行员数， m 是边数

空间复杂度： $(O(n + m))$

伪代码

```
函数 maxMatching(ability):
    n = ability 的行数
    匹配 = [-1] * n  // 记录每个飞机的匹配飞行员
    配对数 = 0

    对于 每个飞行员 pilot 从 0 到 n-1:
        访问集合 = 空集合
        如果 寻找增广路径(pilot, ability, 匹配, 访问集合):
            配对数 += 1

    返回 配对数

函数 寻找增广路径(pilot, ability, 匹配, 访问集合):
    对于 每架飞机 plane 从 0 到 n-1:
        如果 ability[pilot][plane] == 1 且 plane 不在 访问集合:
            访问集合.添加(plane)
```

```
// 飞机未被匹配，或者飞机的匹配飞行员能找到其他飞机
如果 匹配[plane] == -1 或 寻找增广路径(匹配[plane], ability, 匹配, 访问
集合):

    匹配[plane] = pilot
    返回 True

返回 False
```

代码实现

```
class Solution:
    def maxPairing(self, ability: List[List[int]]) -> int:
        """
        使用匈牙利算法求二分图最大匹配
        """
        n = len(ability)
        # match[j] = i 表示飞机j被飞行员i匹配
        match = [-1] * n

        def dfs(pilot, visited):
            """
            为飞行员pilot寻找增广路径
            """
            for plane in range(n):
                # 能驾驶且未访问过
                if ability[pilot][plane] == 1 and plane not in visited:
                    visited.add(plane)

                    # 飞机未被匹配 或 飞机的匹配飞行员能找到其他飞机
                    if match[plane] == -1 or dfs(match[plane], visited):
                        match[plane] = pilot
                        return True

            return False

        # 对每个飞行员尝试匹配
        matching_count = 0
        for pilot in range(n):
            visited = set()
            if dfs(pilot, visited):
                matching_count += 1

        return matching_count

    def isPerfectMatching(self, ability: List[List[int]]) -> bool:
```

```

"""检查是否存在完美匹配"""
n = len(ability)
return self.maxPairing(ability) == n

```

高级版本：使用最大流算法（Dinic）

```

class MaxFlow:
    """Dinic最大流算法"""
    def __init__(self, n):
        self.n = n
        self.graph = [[] for _ in range(n)]

    def add_edge(self, u, v, cap):
        self.graph[u].append([v, cap, len(self.graph[v])])
        self.graph[v].append([u, 0, len(self.graph[u]) - 1])

    def bfs(self, s, t):
        level = [-1] * self.n
        level[s] = 0
        queue = deque([s])

        while queue:
            v = queue.popleft()
            for i, (to, cap, _) in enumerate(self.graph[v]):
                if level[to] < 0 and cap > 0:
                    level[to] = level[v] + 1
                    queue.append(to)

        return level[t] >= 0, level

    def dfs(self, v, t, f, level, iter):
        if v == t:
            return f
            return f

        for i in range(iter[v], len(self.graph[v])):
            to, cap, rev = self.graph[v][i]
            if level[v] < level[to] and cap > 0:
                d = self.dfs(to, t, min(f, cap), level, iter)
                if d > 0:
                    self.graph[v][i][1] -= d
                    self.graph[to][rev][1] += d
                    return d
            iter[v] += 1

        return 0

```

```

def max_flow(self, s, t):
    flow = 0
    while True:
        reachable, level = self.bfs(s, t)
        if not reachable:
            return flow
        iter = [0] * self.n
        f = 1
        while f > 0:
            f = self.dfs(s, t, float('inf'), level, iter)
            flow += f

class Solution:
    def maxPairingWithFlow(self, ability: List[List[int]]) -> int:
        """使用最大流算法求解"""
        n = len(ability)
        # 0: source, 1..n: pilots, n+1..2n: planes, 2n+1: sink
        flow = MaxFlow(2*n + 2)
        source, sink = 0, 2*n + 1

        # source -> pilots (容量1)
        for i in range(n):
            flow.add_edge(source, i + 1, 1)

        # pilots -> planes (如果能驾驶)
        for i in range(n):
            for j in range(n):
                if ability[i][j] == 1:
                    flow.add_edge(i + 1, n + 1 + j, 1)

        # planes -> sink (容量1)
        for j in range(n):
            flow.add_edge(n + 1 + j, sink, 1)

        return flow.max_flow(source, sink)

```

1.3 二分图多重匹配问题

问题背景

当需要建立多对多的配对关系时（例如多个飞行员配一个飞机，或多个飞机配一个飞行员），使用多重匹配。

问题5: 圆桌问题（模拟题）

问题描述： n 个男生和 m 个女生参加舞会，每个男生要跳 d_1 首舞，每个女生要跳 d_2 首舞。检查是否能安排使得每个人跳完规定数量的舞。

解决方案：

- 左侧每个男生拆成 d_1 个节点，右侧每个女生拆成 d_2 个节点
- 所有男生节点连向所有女生节点（容量为1）
- 求最大流，检查是否等于总舞蹈次数

代码实现

```
class Solution:
    def canArrangeDance(self, n: int, m: int, d1: List[int], d2: List[int]) -> bool:
        """
        检查是否能安排圆桌舞会配对
        """
        # 计算总舞蹈次数
        total_dances = sum(d1)
        if total_dances != sum(d2):
            return False

        # 使用匹配算法
        # 将每个男生的多个舞蹈需求看作多条连接
        male_match = {} # 记录男生的匹配情况
        female_capacity = d2.copy() # 每个女生还能跳的舞蹈数

        # 贪心匹配：对每个男生的每次舞蹈机会
        for boy in range(n):
            for _ in range(d1[boy]):
                # 尝试找一个还有容量的女生
                for girl in range(m):
                    if female_capacity[girl] > 0:
                        female_capacity[girl] -= 1
                        break
                else:
                    # 找不到满足条件的女生
                    return False

        return all(cap == 0 for cap in female_capacity)
```

1.4 LeetCode 9: 方格取数（二分图点权最大独立集）

问题描述

在 $(n \times n)$ 的棋盘上，某些格子有数值。两个格子相邻（上下左右）则不能同时取。求最大能取得的数值总和。

这是一个**棋盘二分**问题：将棋盘黑白二分，相邻格子必定颜色不同。

文字说明

建模思路：

1. 对棋盘进行黑白二分（ $(i+j)\%2 == 0$ 为黑色，其余为白色）
2. 最大独立集 = 所有数值和 - 最小顶点覆盖
3. 最小顶点覆盖转化为最小割
4. 最小割 = 最大流

构图方法：

- Source $\rightarrow$ 黑色格子，容量为黑色格子的值
- 黑色格子 $\rightarrow$ 相邻白色格子，容量为 $(+\infty)$
- 白色格子 $\rightarrow$ Sink，容量为白色格子的值

关键公式：

$$\text{最大独立集} = \text{所有权值和} - \text{最小割}$$

伪代码

```
函数 maxGridValue(grid):
    n = grid的行数
    总和 = 所有格子值的总和

    // 构建网络流图
    source = 0
    sink = 1
    黑色节点起始 = 2
    白色节点起始 = 2 + (n*n/2)

    创建最大流图

    对于 每个黑色格子(i, j):
        添加边 source  $\rightarrow$  黑色节点, 容量为grid[i][j]

        对于 每个相邻的白色格子(ni, nj):
```

添加边 黑色节点 $\rightarrow$ 白色节点, 容量为 ∞

对于 每个白色格子(i,j):

添加边 白色节点 $\rightarrow$ sink, 容量为grid[i][j]

最小割 = 求最大流(source, sink)

返回 总和 - 最小割

代码实现

```
class Solution:
    def maxGridValue(self, grid: List[List[int]]) -> int:
        """
        使用最小割求棋盘最大独立集
        """
        n = len(grid)
        m = len(grid[0])

        # 计算总和
        total_sum = sum(sum(row) for row in grid)

        # 构建网络流
        # 0: source, 1: sink
        # 2...: 黑色格子, 之后是白色格子
        source, sink = 0, 1

        # 使用邻接表表示流网络
        flow_graph = defaultdict(lambda: defaultdict(int))

        def add_flow_edge(u, v, cap):
            flow_graph[u][v] += cap

        black_id = {}
        white_id = {}
        node_id = 2

        # 分配节点ID
        for i in range(n):
            for j in range(m):
                if (i + j) % 2 == 0:
                    black_id[(i, j)] = node_id
                    node_id += 1

        for i in range(n):
            for j in range(m):
```

```

        if (i + j) % 2 == 1:
            white_id[(i, j)] = node_id
            node_id += 1

# 添加边
for i in range(n):
    for j in range(m):
        if (i + j) % 2 == 0: # 黑色
            add_flow_edge(source, black_id[(i, j)], grid[i][j])

            # 黑色 -> 相邻白色
            for di, dj in [(0,1), (0,-1), (1,0), (-1,0)]:
                ni, nj = i + di, j + dj
                if 0 <= ni < n and 0 <= nj < m:
                    add_flow_edge(black_id[(i, j)], white_id[(ni,
nj)], float('inf'))
            else: # 白色
                add_flow_edge(white_id[(i, j)], sink, grid[i][j])

# 计算最大流（最小割）
min_cut = self.maxflow(flow_graph, source, sink, node_id)

return total_sum - min_cut

def maxflow(self, graph, source, sink, n):
    """Edmonds-Karp算法计算最大流"""
    flow = 0

    while True:
        # BFS找增广路径
        parent = [-1] * n
        parent[source] = source
        queue = deque([source])

        while queue and parent[sink] == -1:
            u = queue.popleft()
            for v in graph[u]:
                if parent[v] == -1 and graph[u][v] > 0:
                    parent[v] = u
                    queue.append(v)

        if parent[sink] == -1:
            break

# 找路径上的最小容量
path_flow = float('inf')

```

```

        v = sink
        while v != source:
            u = parent[v]
            path_flow = min(path_flow, graph[u][v])
            v = u

        # 更新容量
        v = sink
        while v != source:
            u = parent[v]
            graph[u][v] -= path_flow
            graph[v][u] += path_flow
            v = u

        flow += path_flow

    return flow

```

1.5 LeetCode 24: 骑士共存（二分图最大独立集）

问题描述

在 $(n \times n)$ 的棋盘上放置国际象棋骑士，使得没有两个骑士能互相攻击。骑士以"日"字形攻击（距离为(2,1)或(1,2)的位置）。求最多能放置多少个骑士。

文字说明

算法思想：

1. 棋盘二分（黑白着色）
2. 骑士的攻击关系形成二分图
3. 最大独立集 = n - 最大匹配

关键观察： 骑士从黑色格跳到白色格（距离为(2,1)或(1,2)保证了这一点）

代码实现

```

class Solution:
    def maxKnights(self, n: int, m: int) -> int:
        """
        n×m棋盘最多放置多少不相互攻击的骑士
        """
        # 棋盘二分

```

```

black_count = (n * m + 1) // 2
white_count = n * m // 2

# 骑士的所有攻击方向
knight_moves = [(2,1), (2,-1), (-2,1), (-2,-1),
                (1,2), (1,-2), (-1,2), (-1,-2)]

# 构建二分图
# 黑色格子 id = i*m + j (if (i+j)%2==0)
# 白色格子 id = i*m + j (if (i+j)%2==1)

match_white = [-1] * (n * m) # white -> black的匹配

def dfs(black_pos, visited):
    i, j = black_pos // m, black_pos % m

    for di, dj in knight_moves:
        ni, nj = i + di, j + dj

        if 0 <= ni < n and 0 <= nj < m and (ni+nj) % 2 == 1:
            white_pos = ni * m + nj
            if white_pos not in visited:
                visited.add(white_pos)

                if match_white[white_pos] == -1 or \
                    dfs(match_white[white_pos], visited):
                    match_white[white_pos] = black_pos
                    return True

    return False

# 匹配数
matching = 0
for i in range(n):
    for j in range(m):
        if (i+j) % 2 == 0: # 黑色
            visited = set()
            if dfs(i*m + j, visited):
                matching += 1

# 最大独立集 = 总格子数 - 最大匹配数
return n * m - matching

```

第二部分：路径覆盖问题

2.1 路径覆盖的基本理论

基本概念

路径覆盖(Path Cover)： 用最少的不相交路径覆盖图的所有顶点。

关键公式：

$$\text{最少路径数} = |V| - \text{最大匹配数}$$

其中 ($|V|$) 是顶点数，最大匹配是指在一个特殊构造的二分图中的最大匹配。

建图方法

对于有向无环图(DAG)：

1. 将每个顶点 v 拆成 v_{in} 和 v_{out}
2. 对原图的每条边 $u \rightarrow v$ ，添加 $u_{out} \rightarrow v_{in}$ ，容量为1
3. 每个顶点 $v_{in} \rightarrow v_{out}$ ，容量为1（确保每个顶点只在一条路径上）

2.1.1 LeetCode 3: 最小路径覆盖

问题描述

给定一个有向无环图(DAG)，用最少的不相交简单路径覆盖所有顶点。

文字说明

算法思想：

1. 对DAG进行路径覆盖转化
2. 如果顶点 u 的出度 > 0 ，则在某条包含 u 的路径中
3. 如果路径条数为 k ，则必然有 $n-k$ 个"衔接点"（出度和入度都 >0 的点）
4. 最少路径数 = $n -$ （最多能衔接的点数） = $n -$ 最大匹配数

伪代码

```
函数 minPathCover(n, edges):  
    // 构造二分图用于求最大匹配  
    // 左侧：顶点的出点  
    // 右侧：顶点的入点  
  
    // 如果原图有边  $u \rightarrow v$ ，则在二分图中连  $u_{out} \rightarrow v_{in}$   
    构建二分图matching_graph
```

```
对于 每条原边(u, v):
    在matching_graph中添加 u -> v

// 求最大匹配
max_matching = 二分图最大匹配(matching_graph)

返回 n - max_matching
```

代码实现

```
class Solution:
    def minPathCover(self, n: int, edges: List[List[int]]) -> int:
        """
        DAG最小路径覆盖 = n - 最大匹配数
        """
        # 构建二分图
        graph = [[] for _ in range(n)]
        for u, v in edges:
            graph[u].append(v)

        # 求最大匹配
        match = [-1] * n

        def dfs(u, visited):
            for v in graph[u]:
                if v not in visited:
                    visited.add(v)
                    if match[v] == -1 or dfs(match[v], visited):
                        match[v] = u
                        return True
            return False

        matching = 0
        for u in range(n):
            visited = set()
            if dfs(u, visited):
                matching += 1

        return n - matching
```

2.2 LeetCode 4: 魔术球（动态路径覆盖）

问题描述

给定无限个柱子（编号从1开始），球从顶部落下。每个柱子上可以放球，但相邻两个球之间的柱子编号必须满足特定关系。求放完所有k个球后需要的最少柱子数。

约束条件： 相邻两个球所在柱子的编号之差不能超过1（在同一柱子上或相邻柱子上）。

文字说明

算法思想： 这是一个动态的路径覆盖问题。每次添加一个球时，尝试放在能连接到前一个球的现有路径的末尾，否则开启新路径。

贪心策略：

1. 维护所有开启的柱子，每个柱子的栈顶是最后一个球的编号
2. 对于新球k，如果存在柱子其栈顶的球编号为k-1或k+1，则可以将k放在该柱子
3. 否则必须开启新柱子

时间复杂度： $(O(k))$

伪代码

```
函数 magicBalls(k):
    柱子列表 = [] // 每个柱子的栈顶球编号

    对于 球 b 从 1 到 k:
        找到 = False

        对于 每个柱子 i:
            如果 |b - 柱子列表[i]| <= 1:
                柱子列表[i] = b
                找到 = True
                break

        如果 没找到:
            柱子列表.添加(b)

    返回 柱子数量
```

代码实现

```
class Solution:
    def magicBalls(self, k: int) -> int:
        """
        动态路径覆盖：贪心放置球到柱子
        """
```

```

"""
poles = [] # 每个柱子的栈顶球编号

for ball in range(1, k + 1):
    placed = False

    # 尝试放在现有柱子上
    for i in range(len(poles)):
        if abs(ball - poles[i]) <= 1:
            poles[i] = ball
            placed = True
            break

    # 需要新柱子
    if not placed:
        poles.append(ball)

return len(poles)

def magicBallsOptimized(self, k: int) -> int:
    """
    使用排序优化查找
    """
    from bisect import bisect_left, bisect_right

    poles = []

    for ball in range(1, k + 1):
        # 二分查找能放入的柱子范围 [ball-1, ball+1]
        left_idx = bisect_left(poles, ball - 1)
        right_idx = bisect_right(poles, ball + 1)

        if left_idx < right_idx:
            # 找到可放入的柱子
            idx = left_idx
            poles[idx] = ball
            poles.sort() # 保持有序
        else:
            # 需要新柱子
            poles.append(ball)
            poles.sort()

    return len(poles)

```

第三部分：费用流问题

3.1 费用流的基本理论

定义与概念

费用流(Cost Flow)：在流网络中，每条边不仅有容量，还有单位流量的费用。目标是找到从源到汇的流量为 f 的流，使得总费用最小。

关键参数：

- 流量 (f)
- 单位费用 ($c(u,v)$)
- 总费用 ($= \sum f_e \times c_e$)

最小费用最大流(Minimum Cost Maximum Flow)：

1. 先求最大流
2. 最大流中费用最小

3.1.1 基于增广路的求解

思路： 使用SPFA或Dijkstra找最小费用增广路，重复增广直到达到目标流量。

算法框架：

1. 初始流为0
2. 重复：
 - 使用SPFA/Dijkstra找从source到sink的最小费用增广路
 - 沿该路增广最大可能流量
 - 累加费用
3. 直到达到目标流量或无增广路

代码模板

```
from collections import defaultdict, deque

class MinCostFlow:
    """最小费用流（使用SPFA）"""

    def __init__(self, n):
        self.n = n
        self.edges = []
        self.graph = [[] for _ in range(n)]
```

```

def add_edge(self, u, v, cap, cost):
    """添加有向边"""
    self.graph[u].append(len(self.edges))
    self.edges.append([v, cap, cost])
    self.graph[v].append(len(self.edges))
    self.edges.append([u, 0, -cost]) # 反向边

def spfa(self, s, t):
    """SPFA找最小费用路径"""
    dist = [float('inf')] * self.n
    dist[s] = 0
    parent = [-1] * self.n
    in_queue = [False] * self.n
    queue = deque([s])
    in_queue[s] = True

    while queue:
        u = queue.popleft()
        in_queue[u] = False

        for edge_idx in self.graph[u]:
            v, cap, cost = self.edges[edge_idx]
            if cap > 0 and dist[u] + cost < dist[v]:
                dist[v] = dist[u] + cost
                parent[v] = edge_idx
                if not in_queue[v]:
                    queue.append(v)
                    in_queue[v] = True

    if dist[t] == float('inf'):
        return None, None

    # 回溯找流量
    flow = float('inf')
    v = t
    while v != s:
        edge_idx = parent[v]
        flow = min(flow, self.edges[edge_idx][1])
        v = self.edges[edge_idx ^ 1][0]

    return flow, dist[t]

def min_cost_flow(self, s, t, max_flow):
    """
    求最小费用流

```

```

:param s: 源点
:param t: 汇点
:param max_flow: 目标流量
:return: (总流量, 最小费用)
"""
total_cost = 0
total_flow = 0

while total_flow < max_flow:
    flow, cost = self.spfa(s, t)
    if flow is None:
        break

    # 确定增广流量
    add_flow = min(flow, max_flow - total_flow)

    # 更新边的容量
    v = t
    while v != s:
        edge_idx = parent[v]
        self.edges[edge_idx][1] -= add_flow
        self.edges[edge_idx ^ 1][1] += add_flow
        v = self.edges[edge_idx ^ 1][0]

    total_flow += add_flow
    total_cost += add_flow * cost

return total_flow, total_cost

```

3.2 LeetCode 10: 餐巾计划

问题描述

一家餐厅有 n 天的营业期，第 i 天需要 r_i 条餐巾。餐巾可以：

1. 购买：新餐巾1元/条
2. 洗涤：脏餐巾送洗需要 d 天，费用为 p 元/条

求满足每天需求的最小成本。

文字说明

建模思路：

1. 将时间分为 n 天，每天分为"需求"和"供应"两个阶段
2. 新购餐巾：source $\rightarrow$ day i 的供应，容量 ∞ ，费用为1
3. 继承餐巾：day i 的需求 $\rightarrow$ day $i+1$ 的供应，容量 ∞ ，费用0
4. 洗涤：day i 的需求 $\rightarrow$ day $i+d$ 的供应，容量 ∞ ，费用 p
5. 最终需求：day i 的供应 $\rightarrow$ sink，容量 r_i ，费用0

伪代码

```
函数 napkinPlan(n, r, p, d):
    创建最小费用流图
    source = 0, sink = 2n+1

    // day_need[i] = i, day_supply[i] = n+i

    // 源  $\rightarrow$  购买新餐巾
    add_edge(source, day_supply[i],  $\infty$ , 1) for all i

    // 继承未用餐巾
    add_edge(day_supply[i], day_need[i],  $\infty$ , 0) for all i

    // 继承到下一天
    add_edge(day_need[i], day_supply[i+1],  $\infty$ , 0) for i < n-1

    // 洗涤
    add_edge(day_need[i], day_supply[i+d],  $\infty$ , p) for i+d < n

    // 满足需求
    add_edge(day_need[i], sink, r[i], 0) for all i

    return minCostFlow(source, sink, sum(r))
```

代码实现

```
class Solution:
    def napkinPlan(self, n: int, r: List[int], p: int, d: int) -> int:
        """
        使用费用流求餐巾计划最小成本
        """
        # 节点编号
        # 0: source
        # 1..n: day_need
        # n+1..2n: day_supply
        # 2n+1: sink
```

```

source, sink = 0, 2*n + 1
mcf = MinCostFlow(2*n + 2)

# 购买新餐巾
for i in range(n):
    mcf.add_edge(source, n + 1 + i, float('inf'), 1)

# 未用餐巾继承到该天需求
for i in range(n):
    mcf.add_edge(n + 1 + i, 1 + i, float('inf'), 0)

# 未用餐巾继承到下一天
for i in range(n - 1):
    mcf.add_edge(1 + i, n + 1 + (i + 1), float('inf'), 0)

# 洗涤
for i in range(n):
    if i + d < n:
        mcf.add_edge(1 + i, n + 1 + (i + d), float('inf'), p)

# 满足需求
for i in range(n):
    mcf.add_edge(1 + i, sink, r[i], 0)

total_flow, min_cost = mcf.min_cost_flow(source, sink, sum(r))
return min_cost

```

3.3 LeetCode 17: 运输问题（经典费用流）

问题描述

有 m 个工厂，产量为 s_i ；有 n 个商店，需求为 d_j 。从工厂 i 到商店 j 的运输费用为 $c[i][j]$ 。求最小总成本。

这是一个平衡运输问题（产量总和 = 需求总和）。

文字说明

建模思路：

1. 创建源点和汇点
2. $\text{source} \rightarrow$ 每个工厂 i ，容量 s_i ，费用0
3. 工厂 $i \rightarrow$ 商店 j ，容量 ∞ ，费用 $c[i][j]$

4. 商店 $j \rightarrow$ sink, 容量 d_j , 费用0

时间复杂度: $(O(n \times m \times \text{流量} \times \log n))$

代码实现

```
class Solution:
    def transportProblem(self, m: int, n: int,
                        supply: List[int],
                        demand: List[int],
                        cost: List[List[int]]) -> int:
        """
        运输问题最小费用流
        """
        source, sink = 0, m + n + 1
        mcf = MinCostFlow(m + n + 2)

        # source -> factories
        for i in range(m):
            mcf.add_edge(source, 1 + i, supply[i], 0)

        # factories -> shops
        for i in range(m):
            for j in range(n):
                mcf.add_edge(1 + i, 1 + m + j, float('inf'), cost[i][j])

        # shops -> sink
        for j in range(n):
            mcf.add_edge(1 + m + j, sink, demand[j], 0)

        total_flow, min_cost = mcf.min_cost_flow(source, sink, sum(supply))

        # 检查是否完美匹配
        if total_flow != sum(supply):
            return -1 # 无可行解

        return min_cost
```

第四部分：最大权闭合图问题

4.1 最大权闭合图的基本理论

定义

闭合图(Closed Set): 有向图的一个顶点子集C, 其中任何从C内顶点出发的边都指向C内的顶点。

最大权闭合图: 权值和最大的闭合子集。

4.1.1 转化为最小割问题

关键结论:

最大权闭合图的权值 = 所有正权顶点的权值和 - 最小割

建图方法:

1. 对每个顶点v:
 - 如果 $w(v) > 0$, 添加 $\text{source} \rightarrow v$, 容量 $w(v)$
 - 如果 $w(v) < 0$, 添加 $v \rightarrow \text{sink}$, 容量 $|w(v)|$
 2. 对每条原有的边 (u,v) , 添加 $u \rightarrow v$, 容量 ∞
 3. 求最小割
-

4.2 LeetCode 2: 太空飞行计划

问题描述

有n个实验和m件仪器。每个实验j有价值 v_j , 每件仪器i有成本 c_i 。

若要进行实验j, 则必须购买实验j所需的所有仪器。

求最大的实验价值 - 仪器成本。

文字说明

建模思路:

1. 实验为正权顶点 (权值为价值)
2. 仪器为负权顶点 (权值为-成本)
3. 如果实验j需要仪器i, 则添加边 $j \rightarrow i$
4. 求最大权闭合图

直观理解: 选择一个实验集合, 自动会选择这些实验所需的所有仪器。对应的利益 = 总实验价值 - 总仪器成本。

伪代码

```

函数 spaceFlightPlan(n, m, experimentValue, instrumentCost, need):
    // experimentValue: 实验价值数组
    // instrumentCost: 仪器成本数组
    // need[j]: 实验j需要的仪器列表

    source = 0, sink = 1
    节点编号: experiments: 2..n+1, instruments: n+2..n+m+1

    创建网络流图

    正权值和 = 0
    对于 每个实验 j:
        if experimentValue[j] > 0:
            add_edge(source, 2+j, experimentValue[j])
            正权值和 += experimentValue[j]

        对于 j需要的每件仪器 i:
            add_edge(2+j, n+2+i, ∞)

    对于 每件仪器 i:
        if instrumentCost[i] > 0:
            add_edge(n+2+i, sink, instrumentCost[i])

    最小割 = maxflow(source, sink)
    返回 正权值和 - 最小割

```

代码实现

```

class Solution:
    def spaceFlightPlan(self, n: int, m: int,
                        experimentValue: List[int],
                        instrumentCost: List[int],
                        need: List[List[int]]) -> int:
        """
        最大权闭合图: 太空飞行计划
        """
        # 节点编号
        # 0: source, 1: sink
        # 2..n+1: experiments
        # n+2..n+m+1: instruments

        source, sink = 0, 1
        total_nodes = n + m + 2

        # 使用Dinic算法

```

```
flow = Dinic(total_nodes)

# 正权值和
pos_sum = 0

# 添加实验节点
for j in range(n):
    if experimentValue[j] > 0:
        flow.add_edge(source, 2 + j, experimentValue[j])
        pos_sum += experimentValue[j]

# 实验j -> 所需仪器
for i in need[j]:
    flow.add_edge(2 + j, n + 2 + i, float('inf'))

# 添加仪器节点
for i in range(m):
    flow.add_edge(n + 2 + i, sink, instrumentCost[i])

# 计算最小割
min_cut = flow.max_flow(source, sink)

return pos_sum - min_cut
```

第五部分：分层图与状态空间问题

5.1 分层图的基本概念

分层图(Layered Graph)： 将具有某种状态属性的图展开成多层，每层代表一个特定的状态，层间的边表示状态转移。

应用场景：

- 时间相关的最短路问题
- 有限资源的路径问题
- 状态受限的流问题

5.2 LeetCode 15: 汽车加油行驶

问题描述

在城市网络中行驶，每条路需要特定的油耗。汽车油箱容量有限，需要找从起点到终点的最短路。油可以随时加满（假设所有城市都有油站）。

文字说明

建模思路：

1. 分层图：每层代表一个油量状态 (x, y, fuel)
2. 从(x1, y1)移动到(x2, y2)需要cost油
3. 在城市加油：(x, y, fuel) → (x, y, fuel_max)，费用为加油成本

状态转移：

- 移动到相邻城市（油量减少）
- 加油（油量增加）

代码实现

```
class Solution:
    def carFueling(self, n: int, fuel_tank: int,
                  graph: List[List[tuple]],
                  start: int, end: int) -> int:
        """
        分层图最短路：汽车加油行驶

        graph[i]：城市i的相邻城市列表，每个元素为(邻接城市, 油耗)
        """
        # 状态：(city, fuel_level)
        # 距离：经过的油泵数量（或成本）

        dist = {}
        dist[(start, fuel_tank)] = 0

        heap = [(0, start, fuel_tank)]

        while heap:
            d, city, fuel = heapq.heappop(heap)

            if city == end:
                return d

            if (city, fuel) in dist and dist[(city, fuel)] < d:
                continue

            # 尝试移动到相邻城市
```

```

        for next_city, cost in graph[city]:
            if fuel >= cost:
                new_fuel = fuel - cost
                if (next_city, new_fuel) not in dist or \
                    dist[(next_city, new_fuel)] > d:
                    dist[(next_city, new_fuel)] = d
                    heapq.heappush(heap, (d, next_city, new_fuel))

    # 在本城市加油
    if fuel < fuel_tank:
        new_fuel = fuel_tank
        pump_cost = 1 # 加一次油的成本
        if (city, new_fuel) not in dist or \
            dist[(city, new_fuel)] > d + pump_cost:
            dist[(city, new_fuel)] = d + pump_cost
            heapq.heappush(heap, (d + pump_cost, city, new_fuel))

    return -1 # 无可行路径

```

5.3 LeetCode 14: 孤岛营救（状态压缩）

问题描述

在 $n \times n$ 网格上，有多个钥匙和门。一个人从起点出发，需要经过所有需要的门到达终点。

求最短路径长度。

文字说明

建模思路：

1. 状态：(x, y, 钥匙集合)
2. 钥匙集合用位掩码表示
3. 使用BFS在这个三维状态空间中搜索

代码实现

```

class Solution:
    def shortestPathToGetAllKeys(self, grid: List[str]) -> int:
        """
        状态压缩BFS: 孤岛营救
        """
        m, n = len(grid), len(grid[0])

```

```

# 找起点和所有钥匙位置
start = None
keys_count = 0
for i in range(m):
    for j in range(n):
        if grid[i][j] == '@':
            start = (i, j)
        elif 'a' <= grid[i][j] <= 'f':
            keys_count += 1

# BFS
queue = deque([(start[0], start[1], 0, 0)]) # (x, y, keys_mask,
steps)
visited = set()
visited.add((start[0], start[1], 0))

directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
target_mask = (1 << keys_count) - 1

while queue:
    x, y, keys, steps = queue.popleft()

    if keys == target_mask:
        return steps

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < m and 0 <= ny < n:
            cell = grid[nx][ny]

            # 检查是否能通过
            if cell == '#':
                continue

            new_keys = keys

            # 收集钥匙
            if 'a' <= cell <= 'f':
                new_keys |= (1 << (ord(cell) - ord('a')))

            # 检查门
            if 'A' <= cell <= 'F':
                if not (keys & (1 << (ord(cell) - ord('A')))):
                    continue

```

```
        if (nx, ny, new_keys) not in visited:
            visited.add((nx, ny, new_keys))
            queue.append((nx, ny, new_keys, steps + 1))

    return -1
```

第六部分：特殊算法题型

6.1 LeetCode 8: 机器人路径规划（IDA* 搜索）

问题描述

机器人在障碍物环境中规划路径，避免碰撞，找最短且平滑的路径。

文字说明

IDA (Iterative Deepening A):

1. 结合深度优先搜索的空间效率和启发式搜索的效率
2. 设定深度限制，逐次增加
3. 在限制深度内使用A*搜索

代码框架

```
class Solution:
    def robotPathPlanning(self, grid: List[str],
                          start: tuple, goal: tuple) -> List[tuple]:
        """
        使用IDA*进行路径规划
        """
        def heuristic(pos):
            """曼哈顿距离启发函数"""
            return abs(pos[0] - goal[0]) + abs(pos[1] - goal[1])

        def search(path, g_cost, limit):
            """IDA*搜索"""
            curr = path[-1]
            f = g_cost + heuristic(curr)

            if f > limit:
                return f
```

```

    if curr == goal:
        return 0 # 找到解

    min_limit = float('inf')

    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        next_pos = (curr[0] + dx, curr[1] + dy)
        if next_pos not in path and is_valid(next_pos, grid):
            path.append(next_pos)
            result = search(path, g_cost + 1, limit)

            if result == 0:
                return 0 # 找到解

            min_limit = min(min_limit, result)
            path.pop()

    return min_limit

def is_valid(pos, grid):
    """检查位置是否有效"""
    x, y = pos
    if 0 <= x < len(grid) and 0 <= y < len(grid[0]):
        return grid[x][y] == '.'
    return False

# 迭代深化A*
limit = heuristic(start)
path = [start]

while True:
    result = search(path, 0, limit)
    if result == 0:
        return path
    if result == float('inf'):
        return [] # 无解
    limit = result

```

6.2 LeetCode 21: 最长k可重区间集

问题描述

给定若干区间和一个数k，从这些区间中选择不超过k条（可重复），使得不相交且总长度最大。

文字说明

建模思路：

1. 离散化区间端点
2. 对于每个区间，允许选择最多k次
3. 使用费用流：
 - source → 每个区间，容量k，费用-长度（求最大）
 - 处理时间轴上的冲突

代码框架

```
class Solution:
    def maxIntervalSet(self, intervals: List[tuple], k: int) -> int:
        """
        最长k可重区间集
        """
        # 离散化
        points = set()
        for start, end in intervals:
            points.add(start)
            points.add(end)

        points = sorted(points)
        point_map = {p: i for i, p in enumerate(points)}

        # 使用动态规划
        # dp[i][j] = 在点i处，最多用j个区间的最大长度

        dp = {}

        def solve(point_idx, remaining):
            if point_idx == len(points) or remaining == 0:
                return 0

            if (point_idx, remaining) in dp:
                return dp[(point_idx, remaining)]

            # 选择不包含当前点的区间
            result = solve(point_idx + 1, remaining)

            # 选择包含当前点的区间
            curr_point = points[point_idx]
            for start, end in intervals:
                if start == curr_point:
```

```
        length = end - start
        result = max(result,
                      length + solve(point_map.get(end,
                                                remaining - 1))

        dp[(point_idx, remaining)] = result
    return result

return solve(0, k)
```

总结与实践建议

解题流程

1. 问题识别

- 二分图最大匹配问题
- 路径覆盖问题
- 费用流问题
- 最大权闭合图问题
- 分层图问题

2. 建模

- 确定图的顶点和边
- 设定容量和费用
- 识别源点和汇点

3. 算法选择

- 基础匹配 → 匈牙利算法
- 最大流 → Dinic或Ford-Fulkerson
- 最小费用流 → SPFA或Dijkstra

4. 实现优化

- 使用邻接表表示图
- 实现快速的BFS/SPFA
- 考虑内存和时间限制

常见陷阱

1. **容量和费用方向** - 确保边的方向正确
2. **反向边** - 费用流中的反向边费用应为负
3. **初始化** - 确保所有数据结构初始化正确

4. 边界情况 - 检查 $n=1$ 、无解等特殊情况

扩展阅读

- 最大流算法：Edmonds-Karp, Dinic, Push-Relabel
- 最小费用流：Successive Shortest Paths, Cycle Cancelling
- 高级应用：线性规划的对偶理论

本文档提供了网络流和图论问题的完整解答框架。通过理解每个问题的建模方式和算法选择，可以应对大多数竞赛问题。